

Assignment__2 (Functions & Oops)

Que-1 Explain the importance of functions

Functions in Python programming play a crucial role and are important for several reasons, and I'll explain this in easy language

Simplifying Tasks: Functions are like helpers that you can create to do specific jobs. Instead of writing the same code again and again, you can make a function to do it once. This makes your program shorter and easier to understand

Keeping Things Organized: In a big program, there can be lots of code. Functions help keep things organized. You give each job a name and put the code for that job inside the function. So, you know where to find things

Reusability: Once you create a function, you can use it in different parts of your program whenever you need that job done. This saves time, reduces mistakes, and makes your code more efficient

Collaboration: When working with others, functions make it easier to share and understand the code. You can tell your friends, "Use this function to do that task," and they'll understand what it does.

Que-2 Write a basic function to greet students

Function to greet students

```
def greet_students(name):  
    """  
    This function takes a student's name as input  
    and prints a greeting message.  
    """  
    print(f"Hello, {name}! Welcome to the class.")
```

Example usage

```
greet_students("John")  
greet_students("Alice")
```

```
Hello, John! Welcome to the class.  
Hello, Alice! Welcome to the class.
```

Que-3 What is the difference between print and return statements?

```
def example_with_print():
    print("This is printed")

def example_with_return():
    return "This is returned"

example_with_print() # Outputs to the console
result = example_with_return() # Stored in 'result'
print(result) # Outputs "This is returned"
```

```
This is printed
This is returned
```

*# Que-4 What are *args and **kwargs?*

```
def demo_args_kwargs(*args, **kwargs):
    print("Positional arguments:", args)
    print("Keyword arguments:", kwargs)

demo_args_kwargs(1, 2, 3, name="Alice", age=25)
```

```
Positional arguments: (1, 2, 3)
Keyword arguments: {'name': 'Alice', 'age': 25}
```

```
numbers = iter([1, 2, 3]) # Create an iterator
print(next(numbers)) # 1
print(next(numbers)) # 2
```

```
1
2
```

```
def generate_squares(n):
    for i in range(1, n + 1):
        yield i ** 2
for square in generate_squares(5):
    print(square)
```

```
1
4
9
16
25
```

```
def generate_palindromes(n):
    for i in range(1, n + 1):
        if str(i) == str(i)[::-1]:
            yield i
for palindrome in generate_palindromes(150):
```

```
print(palindrome)
```

```
1
2
3
4
5
6
7
8
9
11
22
33
44
55
66
77
88
99
101
111
121
131
141
```

```
def generate_evens(n):
    for i in range(2, n + 1, 2):
        yield i
for even in generate_evens(10):
    print(even)
```

```
2
4
6
8
10
```

```
def generate_powers_of_two(n):
    for i in range(n):
        yield 2 ** i
for power in generate_powers_of_two(5):
    print(power)
```

```
1
2
4
8
16
```

```

def is_prime(num):
    if num < 2:
        return False
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            return False
    return True

def generate_primes(n):
    for i in range(2, n + 1):
        if is_prime(i):
            yield i
for prime in generate_primes(20):
    print(prime)

2
3
5
7
11
13
17
19

square = lambda x: x ** 2

# Example usage
num = 5
print(f"The square of {num} is {square(num)}")

The square of 5 is 25

sum_two_numbers = lambda a, b: a + b
print(sum_two_numbers(5, 3)) # Output: 8

8

is_even = lambda x: "Even" if x % 2 == 0 else "Odd"
print(is_even(5)) # Output: Odd

Odd

concat_strings = lambda s1, s2: s1 + s2
print(concat_strings("Hello, ", "World!")) # Output: Hello, World!

Hello, World!

max_three = lambda a, b, c: max(a, b, c)
print(max_three(5, 12, 7)) # Output: 12

12

```

```
numbers = [1, 2, 3, 4, 5, 6]
squares_of_evens = [x**2 for x in numbers if x % 2 == 0]
print(squares_of_evens) # Output: [4, 16, 36]
```

```
[4, 16, 36]
```

```
numbers = [-1, 2, -3, 4]
product = 1
for num in numbers:
    if num > 0:
        product *= num
print(product) # Output: 8
```

```
8
```

```
numbers = [1, 2, 3, 4, 5]
doubled_odds = [x * 2 for x in numbers if x % 2 != 0]
print(doubled_odds) # Output: [2, 6, 10]
```

```
[2, 6, 10]
```

```
numbers = [1, 2, 3, 4, 5] # Example list
sum_of_cubes = sum(map(lambda x: x**3, numbers))
print(f"Sum of cubes: {sum_of_cubes}")
```

```
Sum of cubes: 225
```

```
numbers = [2, 3, 4, 5, 6, 7, 8]
primes = list(filter(is_prime, numbers))
print(primes) # Output: [2, 3, 5, 7]
```

```
[2, 3, 5, 7]
```

```
# Lambda function to add two numbers
sum_lambda = lambda a, b: a + b
```

```
# Example usage
```

```
num1 = 5
num2 = 10
result = sum_lambda(num1, num2)
```

```
print(f"The sum of {num1} and {num2} is {result}")
```

```
The sum of 5 and 10 is 15
```

```
# Lambda function to calculate the square of a number
square_lambda = lambda x: x ** 2
```

```
# Example usage
```

```
num = 6
```

```
result = square_lambda(num)
```

```
print(f"The square of {num} is {result}")
```

The square of 6 is 36

```
# Lambda function to check if a number is even or odd
```

```
even_or_odd = lambda x: "Even" if x % 2 == 0 else "Odd"
```

```
# Example usage
```

```
num = 7
```

```
result = even_or_odd(num)
```

```
print(f"The number {num} is {result}.")
```

The number 7 is Odd.

```
# Lambda function to concatenate two strings
```

```
concat_lambda = lambda str1, str2: str1 + str2
```

```
# Example usage
```

```
string1 = "Hello, "
```

```
string2 = "World!"
```

```
result = concat_lambda(string1, string2)
```

```
print(f"Concatenated string: {result}")
```

Concatenated string: Hello, World!

```
# Lambda function to find the maximum of three numbers
```

```
max_lambda = lambda a, b, c: max(a, b, c)
```

```
# Example usage
```

```
num1 = 10
```

```
num2 = 25
```

```
num3 = 15
```

```
result = max_lambda(num1, num2, num3)
```

```
print(f"The maximum of {num1}, {num2}, and {num3} is {result}")
```

The maximum of 10, 25, and 15 is 25

Encapsulation is one of the four fundamental principles of Object-Oriented Programming (OOP), along with Abstraction, Inheritance, and Polymorphism. It refers to the bundling of data (variables) and methods (functions) that operate on that data into a single unit (class), while restricting direct access to some details.

Key Concepts of Encapsulation:

Data Hiding: Prevents direct modification of data by making variables private and providing controlled access through methods (getters & setters).

Access Modifiers: Used to define the level of access to `class` members:

- Private (`__variable`): Only accessible within the `class`.
- Protected (`_variable`): Can be accessed within the `class` and subclasses.
- Public (`__variable`): can be accessed `from` anywhere

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance # Private variable

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance
```

When to Use Access Modifiers
Public:

Use when attributes `and` methods are intended to be accessed by everyone (default behavior).
Example: Methods like `get_balance()` `or` attributes like `name`.
Protected:

Use when attributes `and` methods are meant `for` internal use by the `class` `and` its subclasses.
Example: Helper methods `or` internal state like `_calculate_tax()`.
Private:

Use when attributes `and` methods should `not` be accessible `or` modified directly outside the `class` (e.g., sensitive data `or` internal logic).
Example: Passwords, account balances, `or any` data requiring strict control.

Key Points

Python's access modifiers are `not` strict `and` rely on conventions.
Name mangling (`for` private members) `is` a deterrent, `not` a strict restriction.

Use public `for` simplicity, protected `for` internal use, `and` private `for` sensitive data `or` critical methods.

Let me know `if` you'd like additional examples! 😊

Inheritance `is` a fundamental concept `in` Object-Oriented Programming (OOP) that allows a `class` (called the child `class` `or` subclass) to derive properties `and` behaviors (methods) `from` another `class` (called the parent `class` `or` superclass). This promotes code reuse, modularity, `and` maintainability.

Key Features of Inheritance

1-Code Reusability:

The child `class` inherits attributes `and` methods `from` the parent `class`, reducing the need to duplicate code.

2-Hierarchical Structure:

Enables a structured relationship between classes, where common functionality is centralized in the parent class.

3-Extensibility:

The child class can override or extend the functionality of the parent class to suit its specific needs.

4-Improved Maintainability:

Changes made in the parent class propagate to all child classes, making code easier to maintain.

Polymorphism in Python- In OOP, polymorphism refers to an object's capacity to assume several forms.

Simply said, polymorphism enables us to carry out a single activity in a variety of ways

1. Method overloading : Method overloading is the practice of invoking the same method more than once with different parameters. Method overloading is not supported by Python. Even if you overload the method, Python only takes into account the most recent definition. If you overload a method in Python, a `TypeError` will be raised.

2. Method overriding : In Python, method overriding is the process of providing a different implementation for a method that is already defined in the superclass within a subclass. It enables the subclass to define its own version of a method with the same name and parameters as the method in the superclass. When a method is overridden, the subclass implements the method in its own way, which overrides the behaviour defined in the superclass. The subclass can then alter or expand the functionality of the inherited method.

Method overriding in Python occurs when a child class provides a specific implementation of a method that is already defined in its parent class. This allows the child class to modify or completely change the behavior of the parent class's method to suit its own needs.

Key Features of Method Overriding

1-Same Method Name:

The overridden method in the child class must have the same name as the method in the parent class.

2-Same Parameters:

The signature of the overridden method (number and type of parameters) should match that of the parent method.

3-Dynamic Binding:

At runtime, the method of the object's actual class is called, not the method of its reference class.

4-Use of super():

The super() function can be used to call the parent class's version of the overridden method.

Define the parent class

```
class Animal:
    def make_sound(self):
        print("Generic animal sound")
```

Define the child class

```
class Dog(Animal):
    def make_sound(self):
        print("Woof!")
```

Create objects and demonstrate method overriding

```
animal = Animal()
animal.make_sound()  # Output: Generic animal sound
```

```
dog = Dog()
dog.make_sound()  # Output: Woof!
```

```
Generic animal sound
Woof!
```

Define the parent class

```
class Animal:
    def make_sound(self):
        print("Generic animal sound")

    def move(self):
        print("Animal moves")
```

Define the child class

```
class Dog(Animal):
    def make_sound(self):
        print("Woof!")

    def move(self):
        print("Dog runs")
```

Create objects and demonstrate method overriding

```
animal = Animal()
animal.move()  # Output: Animal moves
```

```

dog = Dog()
dog.move() # Output: Dog runs

Animal moves
Dog runs

# Define the parent class Animal
class Animal:
    def make_sound(self):
        print("Generic animal sound")

    def move(self):
        print("Animal moves")

# Define the class Mammal
class Mammal:
    def reproduce(self):
        print("Giving birth to live young.")

# Define the class Dog inheriting from Animal
class Dog(Animal):
    def make_sound(self):
        print("Woof!")

    def move(self):
        print("Dog runs")

# Define the class DogMammal inheriting from both Dog and Mammal
class DogMammal(Dog, Mammal):
    pass

# Create objects and demonstrate behavior
dog_mammal = DogMammal()
dog_mammal.make_sound() # Output: Woof!
dog_mammal.move() # Output: Dog runs
dog_mammal.reproduce() # Output: Giving birth to live young.

Woof!
Dog runs
Giving birth to live young.

# Define the class Animal
class Animal:
    def make_sound(self):
        print("Generic animal sound")

    def move(self):
        print("Animal moves")

# Define the class Dog inheriting from Animal
class Dog(Animal):

```

```

def make_sound(self):
    print("Woof!")

def move(self):
    print("Dog runs")

# Define the class GermanShepherd inheriting from Dog
class GermanShepherd(Dog):
    def make_sound(self):
        print("Bark!") # Overriding the make_sound method

# Create objects and demonstrate behavior
dog = Dog()
dog.make_sound() # Output: Woof!

german_shepherd = GermanShepherd()
german_shepherd.make_sound() # Output: Bark!

Woof!
Bark!

# Define the Animal class with a constructor
class Animal:
    def __init__(self, name, age):
        self.name = name # Initialize the name of the animal
        self.age = age # Initialize the age of the animal

    def make_sound(self):
        print(f"{self.name} makes a sound!")

    def move(self):
        print(f"{self.name} moves.")

# Define the Dog class with its own constructor
class Dog(Animal):
    def __init__(self, name, age, breed):
        super().__init__(name, age) # Call the parent class
        constructor
        self.breed = breed # Initialize the breed of the dog

    def make_sound(self):
        print(f"{self.name} the {self.breed} barks!")

    def move(self):
        print(f"{self.name} the {self.breed} runs!")

# Create objects and demonstrate behavior
animal = Animal("Generic Animal", 5)
animal.make_sound() # Output: Generic Animal makes a sound!
animal.move() # Output: Generic Animal moves.

```

```
dog = Dog("Buddy", 3, "Golden Retriever")
dog.make_sound()      # Output: Buddy the Golden Retriever barks!
dog.move()             # Output: Buddy the Golden Retriever runs!
```

```
Generic Animal makes a sound!
Generic Animal moves.
Buddy the Golden Retriever barks!
Buddy the Golden Retriever runs!
```

Abstraction **is** one of the key principles of Object-Oriented Programming (OOP). It refers to the concept of hiding **complex** implementation details **and** showing only the necessary features **or** functionality to the user. The main purpose of abstraction **is** to simplify the interaction **with** objects, allowing users to focus on high-level operations without needing to know the underlying details.

In simpler terms:

Abstraction allows you to define what a system should do without specifying how it does it.

How **is** Abstraction Implemented **in** Python?

In Python, abstraction can be implemented using abstract classes **and** abstract methods.

1. Abstract Classes

An abstract **class is a class** that cannot be instantiated (i.e., you cannot create an **object** directly **from** it). It serves **as** a blueprint **for** other classes. Abstract classes can define abstract methods, which must be implemented by subclasses.

2. Abstract Methods

An abstract method **is** a method that **is** declared **in** the abstract **class** but does **not** have **any** implementation. Subclasses must override **and** implement these methods. This enforces a contract that ensures subclasses provide their specific implementation.

In Python, abstraction **is** implemented using the `abc` (Abstract Base Class) module.

Abstraction **is** one of the core principles of Object-Oriented Programming (OOP), **and** it plays a crucial role **in** simplifying **complex** systems. It allows us to focus on what an **object** does, rather than how it does it. Here's why abstraction **is** important:

1. Hiding Implementation Details

Abstraction hides the internal workings of objects **and** only exposes what **is** necessary to the user.

Users can interact **with** objects through a simple interface without needing to understand the **complex** logic inside them.

This reduces complexity and allows the programmer to deal with high-level operations instead of being concerned with low-level details. Example: When you drive a car, you don't need to understand how the engine works (internal complexity); you only need to know how to operate the steering wheel, brakes, and pedals (interface).

2. Reduces Complexity

Abstraction allows us to break down a complex system into smaller, more manageable pieces by hiding unnecessary details.

By focusing on essential features and ignoring irrelevant information, you simplify interactions with the system.

Example: Consider an ATM. The user interacts with the screen to check balance or withdraw money, but they don't need to know how the ATM interacts with banks or how transactions are processed.

3. Promotes Code Reusability

Abstraction helps define common interfaces or methods that can be reused across different classes or modules.

Once the abstract class or method is defined, any subclass can reuse and extend it without needing to rewrite common logic.

Example: You might have an abstract Shape class with an abstract method area(). Then, different shapes like Circle, Rectangle, and Triangle can inherit Shape and implement the area() method in their own way, while keeping the same interface.

4. Improves Maintainability

By providing a clear separation between the interface (what an object can do) and the implementation (how the object does it), abstraction makes maintaining and updating code easier.

If you need to change the internal logic of an object (e.g., how a Car works), you can do it without affecting how the user interacts with the object.

You can modify or add new implementations in subclasses without changing the interface provided by the parent class.

Example: If you have a payment gateway system with different methods like PayPal, CreditCard, and ApplePay, all inheriting from a common abstract class PaymentProcessor, you can update each payment method's internal logic independently, without changing how they are used in your system.

5. Enables Polymorphism

Abstraction is a key enabler of polymorphism, where the same interface can be used to represent different types of objects.

Different classes can implement the same abstract methods in their own way, allowing a unified interface while maintaining distinct behavior.

Example: You can have an abstract Vehicle class with a move() method. The method can be implemented differently for Car, Bike, and Boat, but all of them can be treated as Vehicle objects, enabling polymorphic behavior.

6. Helps Design Clear Interfaces

Abstraction helps design clear and consistent interfaces for objects and classes.

This leads to more structured code and helps developers focus on the interaction between objects, rather than the specifics of how they are implemented.

Example: Consider a database system that has a class `DatabaseConnection`. Through abstraction, the user interacts with a common interface to connect to various types of databases (SQL, NoSQL, etc.) without needing to understand the implementation details of each.

7. Supports Flexibility and Extensibility

Abstraction makes the system flexible and extensible because you can add new functionality or change implementations without affecting the existing codebase.

It allows for the introduction of new derived classes that provide specific behavior, making the system adaptable to changing requirements.

Example: If you have an abstract class `NotificationService`, new types of notifications (SMS, Email, Push) can be added without altering the existing code. You only need to create a new subclass and provide the implementation for sending that notification type.

8. Facilitates Clearer Code and Documentation

Abstraction encourages the use of well-defined interfaces, leading to clearer code that is easier to understand.

As only essential details are provided, other developers (or even future you!) can easily understand how to interact with the class and what behavior to expect without delving into implementation specifics.

Aspect	Abstract Method	Regular Method
Implementation	No implementation in the abstract class; must be implemented by subclasses.	Has implementation in the class; can be used as-is or overridden.
Purpose	Enforces a contract that subclasses must follow. Provides functionality that can be reused or overridden.	
Instantiation	Abstract classes containing abstract methods cannot be instantiated.	Can be part of concrete classes that can be instantiated.
Subclass Requirement	Subclasses must implement abstract methods.	Subclasses can use or override regular methods.
Use	Used when you want to enforce a common interface across subclasses.	Used to define common functionality for classes.

Achieving Abstraction with Interfaces in Python

While Python does not have an interface keyword (like Java or C#), you can simulate interfaces using abstract base classes (ABCs). An abstract base class can define abstract methods that must be

implemented by any subclass, thus acting as an interface that enforces a certain structure.

```
from abc import ABC, abstractmethod

# Step 1: Define an abstract base class (interface)
class Vehicle(ABC):
    @abstractmethod
    def start_engine(self):
        pass

    @abstractmethod
    def move(self):
        pass

    @abstractmethod
    def stop(self):
        pass

# Step 2: Define concrete classes implementing the interface

# Car class implementing the Vehicle interface
class Car(Vehicle):
    def start_engine(self):
        print("Starting the car engine.")

    def move(self):
        print("The car is moving.")

    def stop(self):
        print("The car has stopped.")

# Bike class implementing the Vehicle interface
class Bike(Vehicle):
    def start_engine(self):
        print("Starting the bike engine.")

    def move(self):
        print("The bike is moving.")

    def stop(self):
        print("The bike has stopped.")

# Truck class implementing the Vehicle interface
class Truck(Vehicle):
    def start_engine(self):
        print("Starting the truck engine.")

    def move(self):
        print("The truck is moving.")
```

```

    def stop(self):
        print("The truck has stopped.")

# Step 3: Demonstrate polymorphism via the common interface

# Function that works with any vehicle (Polymorphism in action)
def operate_vehicle(vehicle: Vehicle):
    vehicle.start_engine()
    vehicle.move()
    vehicle.stop()

# Create instances of different vehicle types
car = Car()
bike = Bike()
truck = Truck()

# Operate all vehicles using the same interface
print("Operating the Car:")
operate_vehicle(car)

print("\nOperating the Bike:")
operate_vehicle(bike)

print("\nOperating the Truck:")
operate_vehicle(truck)

Operating the Car:
Starting the car engine.
The car is moving.
The car has stopped.

Operating the Bike:
Starting the bike engine.
The bike is moving.
The bike has stopped.

Operating the Truck:
Starting the truck engine.
The truck is moving.
The truck has stopped.

# Polymorphism is a core concept in Object-Oriented Programming (OOP)
that allows different classes to provide different implementations of
the same method, even though the method has the same name. This is
often referred to as method overriding.

# In Python, polymorphism is achieved through method overriding where
a subclass provides its own implementation of a method that is already
defined in its parent (or base) class.

```

```

# Parent class
class Animal:
    def speak(self):
        print("Some generic animal sound")

# Subclass Dog overrides the speak method
class Dog(Animal):
    def speak(self):
        print("Woof!")

# Subclass Cat overrides the speak method
class Cat(Animal):
    def speak(self):
        print("Meow!")

# Demonstrating polymorphism
def make_animal_speak(animal: Animal):
    animal.speak()

# Creating instances of Dog and Cat
dog = Dog()
cat = Cat()

# Calling the same function on different objects (polymorphism in action)
make_animal_speak(dog) # Output: Woof!
make_animal_speak(cat) # Output: Meow!

Woof!
Meow!

# Base class (Parent class)
class Animal:
    def make_sound(self):
        print("Some generic animal sound")

# Subclass (Child class) that overrides the method
class Dog(Animal):
    def make_sound(self):
        print("bho bho!")

# Subclass (Child class) that overrides the method
class monkey(Animal):
    def make_sound(self):
        print("khaw khaw!")

# Creating instances of Dog and Cat
dog = Dog()
monkey = monkey()

# Calling the overridden methods

```

```

dog.make_sound() # Output: Woof!
monkey.make_sound() # Output: Meow!

bho bho!
khaw khaw!

# Base class (Parent class)
class Vehicle:
    def start_engine(self):
        print("Starting the vehicle engine...")

# Subclass 1: Car
class Car(Vehicle):
    def start_engine(self):
        print("Starting the car engine...")

# Subclass 2: Bike
class Bike(Vehicle):
    def start_engine(self):
        print("Starting the bike engine...")

# Subclass 3: Truck
class Truck(Vehicle):
    def start_engine(self):
        print("Starting the truck engine...")

# Demonstrating the overridden methods
# Create objects of each subclass
car = Car()
bike = Bike()
truck = Truck()

# Call the overridden method for each object
car.start_engine() # Output: Starting the car engine...
bike.start_engine() # Output: Starting the bike engine...
truck.start_engine() # Output: Starting the truck engine...

Starting the car engine...
Starting the bike engine...
Starting the truck engine...

class Animal:
    """Base class representing an animal."""

    def __init__(self, name):
        self.name = name

    def make_sound(self):
        """Method to be overridden by subclasses."""
        return "Some generic animal sound"

```

```

class Dog(Animal):
    """Subclass representing a Dog."""

    def make_sound(self):
        return "Woof! Woof!"

class Cat(Animal):
    """Subclass representing a Cat."""

    def make_sound(self):
        return "Meow! Meow!"

class Cow(Animal):
    """Subclass representing a Cow."""

    def make_sound(self):
        return "Moo! Moo!"

# Creating objects of each subclass
dog = Dog("Buddy")
cat = Cat("Whiskers")
cow = Cow("Bessie")

# Calling overridden methods
print(f"{dog.name} says: {dog.make_sound()}")
print(f"{cat.name} says: {cat.make_sound()}")
print(f"{cow.name} says: {cow.make_sound()}")

```

Buddy says: Woof! Woof!
 Whiskers says: Meow! Meow!
 Bessie says: Moo! Moo!

How Polymorphism Improves Code Readability and Reusability
 Polymorphism is a key concept in Object-Oriented Programming (OOP) that allows methods in different classes to have the same name but different implementations. This improves code readability and reusability in several ways:

1. Improves Readability

Consistent method names: Using the same method name across different classes makes code easier to understand.

Reduces unnecessary conditionals: Instead of checking object types manually, polymorphism allows us to call methods dynamically.

2. Enhances Reusability

Same interface for different behaviors: A single function can handle multiple object types, reducing code duplication.

Easier maintenance: Updating a method in a base class automatically applies to all subclasses.

```

class Dog:
    def make_sound(self):
        return "Woof! Woof!"

class Cat:
    def make_sound(self):
        return "Meow! Meow!"

class Car:
    def make_sound(self):
        return "Vroom! Vroom!"

# Function using duck typing
def animal_sound(entity):
    print(entity.make_sound())

# Using different objects with the same method
dog = Dog()
cat = Cat()
car = Car() # Unrelated class but has the same method

animal_sound(dog) # Woof! Woof!
animal_sound(cat) # Meow! Meow!
animal_sound(car) # Vroom! Vroom! (No inheritance, but works)

Woof! Woof!
Meow! Meow!
Vroom! Vroom!

```

Encapsulation in Python

Encapsulation is an object-oriented programming (OOP) principle that restricts direct access to an object's data and allows controlled modification through methods. It helps maintain data integrity and prevents unintended interference.

```

class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner # Public attribute
        self._balance = balance # Protected attribute
        self.__account_number = "123456789" # Private attribute

    # Public method to get balance
    def get_balance(self):
        return self._balance

    # Public method to set balance with validation
    def set_balance(self, amount):
        if amount < 0:
            raise ValueError("Balance cannot be negative")
        self._balance = amount

```

```

    # Private method (only accessible inside the class)
    def __get_account_number(self):
        return self.__account_number

# Creating an object
account = BankAccount("Alice", 1000)

# Accessing public attributes
print(account.owner) # Alice

# Accessing protected attributes (allowed but not recommended)
print(account._balance) # 1000

# Accessing private attributes (Raises an AttributeError)
# print(account.__account_number) # Error: AttributeError

# Using public methods to interact with private/protected data
print(account.get_balance()) # 1000
account.set_balance(1500)
print(account.get_balance()) # 1500

```

Cell In[4], line 2

Encapsulation is an object-oriented programming (OOP) principle that restricts direct access to an object's data and allows controlled modification through methods. It helps maintain data integrity and prevents unintended interference.

^

SyntaxError: unterminated string literal (detected at line 2)

Yes, encapsulation can be bypassed in Python. Python's encapsulation is based on conventions and namespace mangling, rather than strict access control.

Here are some ways encapsulation can be bypassed:

1. ***Direct access to attributes***: Python doesn't have strict access control, so you can access attributes directly using their name.

```

'''
'''

```

```

class MyClass:
    def __init__(self):
        self.__private_attr = 0

```

```

obj = MyClass()
print(obj._MyClass__private_attr) # Accessing private attribute
'''

```

2. ****Namespace mangling****: Python's namespace mangling can be

bypassed by accessing the attribute using its mangled name.

```
```python
class MyClass:
 def __init__(self):
 self.__private_attr = 0

obj = MyClass()
print(obj._MyClass__private_attr) # Accessing private attribute
```
```

3. *Using the `\_\_dict\_\_` attribute*: You can access an object's attributes using the `\_\_dict\_\_` attribute.

```
```
```
class MyClass:
    def *init*(self):
        self.__private_attr = 0

obj = MyClass()
print(obj.__dict*['_MyClass__private_attr']) # Accessing private attribute
```
```

4. \*\*Using the `inspect` module\*\*: You can use the `inspect` module to access an object's attributes.

```
```python
import inspect

class MyClass:
    def __init__(self):
        self.__private_attr = 0

obj = MyClass()
print(inspect.getmembers(obj, lambda a: a.startswith('_MyClass__'))[0][1]) # Accessing private attribute
```
```

Keep in mind that bypassing encapsulation can break the intended behavior of an object and should be done with caution.

```
class BankAccount:
 def __init__(self, initial_balance=0.0):
 self.__balance = initial_balance # Private attribute

 def deposit(self, amount):
 if amount > 0:
 self.__balance += amount
 print(f"Deposited: ${amount:.2f}")
```

```

 else:
 print("Deposit amount must be positive.")

 def withdraw(self, amount):
 if 0 < amount <= self.__balance:
 self.__balance -= amount
 print(f"Withdrew: ${amount:.2f}")
 else:
 print("Invalid withdrawal amount or insufficient funds.")

 def get_balance(self):
 return self.__balance

```

*# Example usage:*

```

account = BankAccount(100.0)
account.deposit(50)
account.withdraw(30)
print(f"Balance: ${account.get_balance():.2f}")

```

Deposited: \$50.00

Withdrew: \$30.00

Balance: \$120.00

```

class Person:
 def __init__(self, name, email):
 self.__name = name # Private attribute
 self.__email = email # Private attribute

 def set_email(self, new_email):
 self.__email = new_email

 def get_email(self):
 return self.__email

```

*# Example usage:*

```

person = Person("John Doe", "john.doe@example.com")
print(f"Email: {person.get_email()}")
person.set_email("new.email@example.com")
print(f"Updated Email: {person.get_email()}")

```

Email: john.doe@example.com

Updated Email: new.email@example.com

Encapsulation is considered a pillar of Object-Oriented Programming (OOP) because it helps in data hiding, security, modularity, and maintainability. Here's why it is fundamental:

### 1. Data Hiding & Security

Encapsulation restricts direct access to an object's data and prevents accidental modification.

Private attributes (`__attribute`) ensure that internal details of a

class remain hidden from outside interference.

## 2. Controlled Access with Methods

Instead of allowing direct access to data, encapsulation provides getter and setter methods for controlled data modification.

Example: A bank account balance should not be modified directly but through deposit() and withdraw() methods.

## 3. Improved Maintainability

Since attributes are hidden and accessed only through specific methods, changes in class implementation do not affect external code.

Example: If the internal representation of an attribute changes, only the class methods need modification, not every place where the object is used.

## 4. Modularity & Code Organization

Encapsulation allows bundling of related data and methods within a single class, making the code more organized and modular.

Each object manages its own state, reducing dependencies between different parts of the program.

## 5. Prevention of Unintended Modifications

Without encapsulation, external code can modify object attributes directly, potentially causing unintended behavior.

Example: Preventing negative balances in a BankAccount class by enforcing validation rules in methods.

## 6. Encapsulation Supports Other OOP Concepts

Abstraction: Encapsulation helps abstract implementation details.

Inheritance & Polymorphism: Encapsulation ensures derived classes access only what's necessary, improving code reuse.

```
person = Person("John Doe", "john.doe@example.com")
print(f"Email: {person.get_email()}")
person.set_email("new.email@example.com")
print(f"Updated Email: {person.get_email()}")
```

```
def message_decorator(func):
 def wrapper(*args, **kwargs):
 print("Before function execution...")
 result = func(*args, **kwargs)
 print("After function execution...")
 return result
 return wrapper
```

```
@message_decorator
def sample_function():
 print("Inside the function!")
```

```
Example usage:
sample_function()
```

```
Email: john.doe@example.com
Updated Email: new.email@example.com
Before function execution...
```

Inside the function!  
After function execution...

Here's how you can modify the decorator to accept arguments and print the function name along with the message:

```
'''
def my_decorator(message):
 def decorator_function(original_function):
 def wrapper_function(*args, **kwargs):
 print(f"{original_function.__name__}: {message}")
 return original_function(*args, **kwargs)
 return wrapper_function
 return decorator_function

@my_decorator("This is a test message")
def add_numbers(a, b):
 return a + b

result = add_numbers(5, 7)
print(result)
'''
```

In this modified version:

- The `my_decorator` function now accepts a `message` argument.
- The `decorator_function` returns the `wrapper_function`, which now prints the original function's name along with the provided message.
- The `wrapper_function` still calls the original function with the original arguments and returns its result.

When you run this code, it will print:

```
'''
add_numbers: This is a test message
12
'''
```

Here's an example where we create two decorators, `decorator1` and `decorator2`, and apply them to a single function, `example_function`. We'll ensure that they execute in the order they are applied:

```
'''
def decorator1(func):
 def wrapper():
 print("Decorator 1: Before calling the function")
 func()
 print("Decorator 1: After calling the function")
 return wrapper

def decorator2(func):
```

```

def wrapper():
 print("Decorator 2: Before calling the function")
 func()
 print("Decorator 2: After calling the function")
 return wrapper

@decorator1
@decorator2
def example_function():
 print("Inside the example function")

example_function()

```

When you run this code, it will print:

```

...
Decorator 1: Before calling the function
Decorator 2: Before calling the function
Inside the example function
Decorator 2: After calling the function
Decorator 1: After calling the function
...

```

As you can see, the decorators execute in the order they are applied:

1. `decorator1` is applied first, so it executes first.
2. `decorator2` is applied second, so it executes second.

The order of execution is determined by the order in which the decorators are applied to the function. The closest decorator to the function is executed first, and then the next closest, and so on.

```

def my_decorator(func):
 def wrapper(*args, **kwargs):
 print("Before calling the function")
 result = func(*args, **kwargs)
 print("After calling the function")
 return result
 return wrapper

@my_decorator
def add_numbers(a, b):
 return a + b

result = add_numbers(5, 7)
print("Result:", result)

```

Before calling the function  
After calling the function  
Result: 12

```
import functools

def my_decorator(func):
 @functools.wraps(func)
 def wrapper(*args, **kwargs):
 print("Before calling the function")
 result = func(*args, **kwargs)
 print("After calling the function")
 return result
 return wrapper

@my_decorator
def add_numbers(a, b):
 """
 Returns the sum of two numbers.

 Args:
 a (int): The first number.
 b (int): The second number.

 Returns:
 int: The sum of a and b.
 """
 return a + b

print(add_numbers.__name__) # Output: add_numbers
print(add_numbers.__doc__) # Output: Returns the sum of two
 numbers...

add_numbers

 Returns the sum of two numbers.

 Args:
 a (int): The first number.
 b (int): The second number.

 Returns:
 int: The sum of a and b.

class Calculator:
 @staticmethod
 def add(num1, num2):
```

```

 """
 Returns the sum of two numbers.

 Args:
 num1 (int or float): The first number.
 num2 (int or float): The second number.

 Returns:
 int or float: The sum of num1 and num2.
 """
 return num1 + num2

```

```

Example usage:
result = Calculator.add(5, 7)
print("Result:", result) # Output: Result: 12

```

Result: 12

```

class Employee:
 employee_count = 0

 def __init__(self, name, age):
 """
 Initializes an Employee object.

 Args:
 name (str): The employee's name.
 age (int): The employee's age.
 """
 self.name = name
 self.age = age
 Employee.employee_count += 1

 @classmethod
 def get_employee_count(cls):
 """
 Returns the total number of employees created.

 Returns:
 int: The total number of employees.
 """
 return cls.employee_count

Example usage:
employee1 = Employee("John Doe", 30)
employee2 = Employee("Jane Smith", 25)
employee3 = Employee("Bob Johnson", 40)

```

```
print("Total employees:", Employee.get_employee_count()) # Output:
Total employees: 3
```

Total employees: 3

```
class StringFormatter:
 @staticmethod
 def reverse_string(input_string):
 """
 Returns the reverse of the input string.

 Args:
 input_string (str): The string to be reversed.

 Returns:
 str: The reversed string.
 """
 return input_string[::-1]

Example usage:
original_string = "Hello, World!"
reversed_string = StringFormatter.reverse_string(original_string)
print("Original string:", original_string)
print("Reversed string:", reversed_string)
```

Original string: Hello, World!  
Reversed string: !dlroW ,olleH

```
import math

class Circle:
 @classmethod
 def calculate_area(cls, radius):
 """
 Calculates the area of a circle given its radius.

 Args:
 radius (float): The radius of the circle.

 Returns:
 float: The area of the circle.
 """
 if radius < 0:
 raise ValueError("Radius cannot be negative")
 return math.pi * (radius ** 2)

Example usage:
radius = 5
```

```
area = Circle.calculate_area(radius)
print(f"The area of the circle with radius {radius} is {area:.2f}")
```

The area of the circle with radius 5 is 78.54

```
class TemperatureConverter:
 @staticmethod
 def celsius_to_fahrenheit(celsius):
 """
 Converts Celsius to Fahrenheit.

 Args:
 celsius (float): Temperature in Celsius.

 Returns:
 float: Temperature in Fahrenheit.
 """
 if celsius < -273.15:
 raise ValueError("Temperature cannot be below absolute
zero")
 return (celsius * 9/5) + 32

Example usage:
celsius = 30
fahrenheit = TemperatureConverter.celsius_to_fahrenheit(celsius)
print(f"{celsius}°C is equal to {fahrenheit}°F")
```

30°C is equal to 86.0°F

The `__str__()` method in Python classes is a special method that returns a string representation of the object. Its purpose is to provide a human-readable representation of the object, which can be useful for:

- Printing objects
- Logging
- Displaying objects in a user interface

Here's an example:

```
...
class Person:
 def __init__(self, name, age):
 self.name = name
 self.age = age

 def __str__(self):
 return f"{self.name}, {self.age} years old"
```

```

Create a Person object
person = Person("John Doe", 30)

Print the object
print(person) # Output: John Doe, 30 years old

The `__len__()` method in Python is a special method that returns
the length (number of items) of an object. It is called when the
built-in `len()` function is used on an object.

Here's an example:

```
class Bookshelf:
 def __init__(self):
 self.books = []

 def add_book(self, title):
 self.books.append(title)

 def __len__(self):
 return len(self.books)

Create a Bookshelf object
bookshelf = Bookshelf()

Add some books
bookshelf.add_book("Book 1")
bookshelf.add_book("Book 2")
bookshelf.add_book("Book 3")

Get the number of books on the bookshelf
num_books = len(bookshelf)
print(f"There are {num_books} books on the bookshelf.") # Output:
There are 3 books on the bookshelf.

```

There are 3 books on the bookshelf.

```

The `__add__()` method in Python is a special method that allows you
to overload the `+` operator for your custom classes. This method is
called when you use the `+` operator on two objects of the same class.

Here's an example:

```
class Vector:
 def __init__(self, x, y):
 self.x = x
 self.y = y

```

```

def __add__(self, other):
 return Vector(self.x + other.x, self.y + other.y)

def __str__(self):
 return f"({self.x}, {self.y})"

Create two Vector objects
v1 = Vector(2, 3)
v2 = Vector(4, 5)

Add the two vectors using the + operator
v3 = v1 + v2

print("v1:", v1) # Output: v1: (2, 3)
print("v2:", v2) # Output: v2: (4, 5)
print("v1 + v2:", v3) # Output: v1 + v2: (6, 8)

v1: (2, 3)
v2: (4, 5)
v1 + v2: (6, 8)

The `__getitem__()` method in Python is a special method that allows
you to overload the indexing operator (`[]`) for your custom classes.
This method is called when you try to access an element of an object
using the indexing syntax.

The purpose of the `__getitem__()` method is to:

- Allow indexing of custom objects
- Enable slicing of custom objects
- Provide a way to access elements of a custom object using the
indexing syntax

class Vector:
 def __init__(self, components):
 self.components = components

 def __getitem__(self, index):
 return self.components[index]

 def __len__(self):
 return len(self.components)

Create a Vector object
v = Vector([1, 2, 3, 4, 5])

Access elements using indexing
print(v[0]) # Output: 1
print(v[1]) # Output: 2

```

```

Slice the vector
print(v[1:3]) # Output: [2, 3]

Get the length of the vector
print(len(v)) # Output: 5

1
2
[2, 3]
5

In Python, the `__iter__()` and `__next__()` methods are used to
create iterators, which allow you to iterate over a sequence of
values.

`__iter__()`:

The `__iter__()` method returns an iterator object that defines the
`__next__()` method. This method is called when you create an iterator
from an object.

`__next__()`:

The `__next__()` method returns the next item from the iterator. If
there are no more items to return, it raises a `StopIteration`
exception.

class MyIterator:
 def __init__(self, data):
 self.data = data
 self.index = 0

 def __iter__(self):
 return self

 def __next__(self):
 if self.index < len(self.data):
 value = self.data[self.index]
 self.index += 1
 return value
 else:
 raise StopIteration

Create an iterator
data = [1, 2, 3, 4, 5]
iterator = MyIterator(data)

Iterate over the data
for value in iterator:
 print(value)

```

1  
2  
3  
4  
5

*# In Python, a getter method is a special method that allows you to access the value of a private attribute of an object. The primary purpose of a getter method is to:*

- # - Encapsulate the internal state of an object*
- # - Control access to an object's attributes*
- # - Provide a way to compute the value of an attribute on the fly*

*# Python provides a built-in decorator called '@property' that allows you to define getter methods in a concise and readable way.*

```
class Person:
 def __init__(self, first_name, last_name):
 self._first_name = first_name
 self._last_name = last_name

 @property
 def full_name(self):
 return f"{self._first_name} {self._last_name}"
```

*# Create a Person object*  
person = Person("John", "Doe")

*# Access the full\_name attribute using the getter method*  
print(person.full\_name) *# Output: John , Doe*

John Doe

*# In Python, a setter method is a special method that allows you to modify the value of a private attribute of an object.*

```
class Person:
 def __init__(self, first_name, last_name):
 self._first_name = first_name
 self._last_name = last_name

 @property
 def full_name(self):
 return f"{self._first_name} {self._last_name}"

 @full_name.setter
 def full_name(self, value):
 first_name, last_name = value.split()
 if not first_name or not last_name:
```

```

 raise ValueError("Invalid full name")
 self._first_name = first_name
 self._last_name = last_name

Create a Person object
person = Person("John", "Doe")

Access the full_name attribute
print(person.full_name) # Output: John Doe

Modify the full_name attribute using the setter method
person.full_name = "Jane Smith"

Access the modified full_name attribute
print(person.full_name) # Output: Jane Smith

John Doe
Jane Smith

The `@property` decorator in Python is used to create getter,
setter, and deleter methods for an attribute of a class. This
decorator allows you to implement encapsulation, which is the concept
of hiding the internal state of an object and only exposing it through
controlled interfaces.

The main purposes of the `@property` decorator are:

- To provide a way to access and modify the internal state of an
object in a controlled manner
- To implement encapsulation and data hiding
- To add functionality to the getter and setter methods, such as
validation or computation

class Person:
 def __init__(self, first_name, last_name):
 self._first_name = first_name
 self._last_name = last_name

 @property
 def full_name(self):
 return f"{self._first_name} {self._last_name}"

 @full_name.setter
 def full_name(self, value):
 first_name, last_name = value.split()
 self._first_name = first_name
 self._last_name = last_name

 @full_name.deleter
 def full_name(self):

```

```

 del self._first_name
 del self._last_name

Create a Person object
person = Person("John", "Doe")

Access the full_name attribute
print(person.full_name) # Output: John Doe

Modify the full_name attribute
person.full_name = "Jane Smith"
print(person.full_name) # Output: Jane Smith

Delete the full_name attribute
del person.full_name
try:
 print(person.full_name)
except AttributeError:
 print("full_name attribute deleted")

John Doe
Jane Smith
full_name attribute deleted

The `@deleter` decorator in Python is used in conjunction with the
`@property` decorator to define a deleter method for a property. A
deleter method is called when the `del` statement is used to delete a
property.

The general syntax for using the `@deleter` decorator is:

```
@property
def property_name(self):
 # Getter method implementation

@property_name.deleter
def property_name(self):
 # Deleter method implementation

class Person:
 def __init__(self, first_name, last_name):
 self._first_name = first_name
 self._last_name = last_name

 @property
 def full_name(self):
 return f"{self._first_name} {self._last_name}"

 @full_name.setter

```

```

def full_name(self, value):
 first_name, last_name = value.split()
 self._first_name = first_name
 self._last_name = last_name

@full_name.deleter
def full_name(self):
 print("Deleting full_name property...")
 del self._first_name
 del self._last_name

Create a Person object
person = Person("John", "Doe")

Access the full_name property
print(person.full_name) # Output: John Doe

Modify the full_name property
person.full_name = "Jane Smith"
print(person.full_name) # Output: Jane Smith

Delete the full_name property
del person.full_name
try:
 print(person.full_name)
except AttributeError:
 print("full_name property deleted")

```

```

Cell In[27], line 10
 @property_name.deleter
 ^

```

IndentationError: expected an indented block after function definition on line 7

*# Encapsulation is a fundamental concept in object-oriented programming (OOP) that binds together the data and the methods that manipulate that data. In Python, encapsulation can be achieved using property decorators.*

*# Property decorators provide a way to implement encapsulation by:*

- # - Hiding the internal state of an object (data encapsulation)*
- # - Controlling access to the internal state through getter and setter methods (access control)*

*# Here's an example showcasing encapsulation using property decorators:*

```

```
class BankAccount:

```

```

def __init__(self, account_number, balance=0):
 self.__account_number = account_number
 self.__balance = balance

@property
def account_number(self):
 return self.__account_number

@property
def balance(self):
 return self.__balance

@balance.setter
def balance(self, value):
 if value < 0:
 raise ValueError("Balance cannot be negative")
 self.__balance = value

Create a BankAccount object
account = BankAccount("1234567890", 1000)

Access the account_number property
print(account.account_number) # Output: 1234567890

Access the balance property
print(account.balance) # Output: 1000

Modify the balance property
account.balance = 500
print(account.balance) # Output: 500

Attempt to set a negative balance
try:
 account.balance = -100
except ValueError as e:
 print(e) # Output: Balance cannot be negative

1234567890
1000
500
Balance cannot be negative

```